Docker Overview

Contents

- Why Docker ?
- What is Docker ?
  - Docker Container
  - Docker Platform Architecture
  - Docker Lice Cycle
  - Docker Eco System
- Benefits/Drawbacks of Docker

Why Docker ?

Market View:
Evolution of IT to Microservice

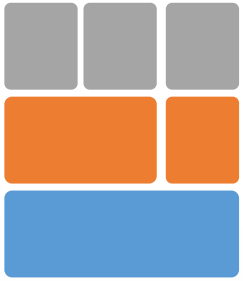
1995

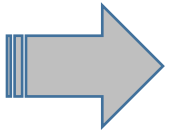
2015

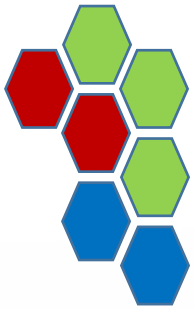
Thick, client-server app on thick client

Thin app on mobile, tablet

Well-defined stack:
- O/S
- Runtime
- Middleware

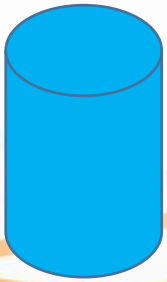






Assembled by developers using best available services

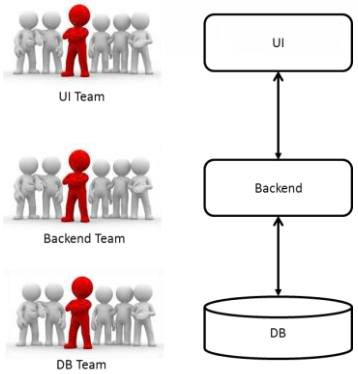
Monolithic Physical Infrastructure



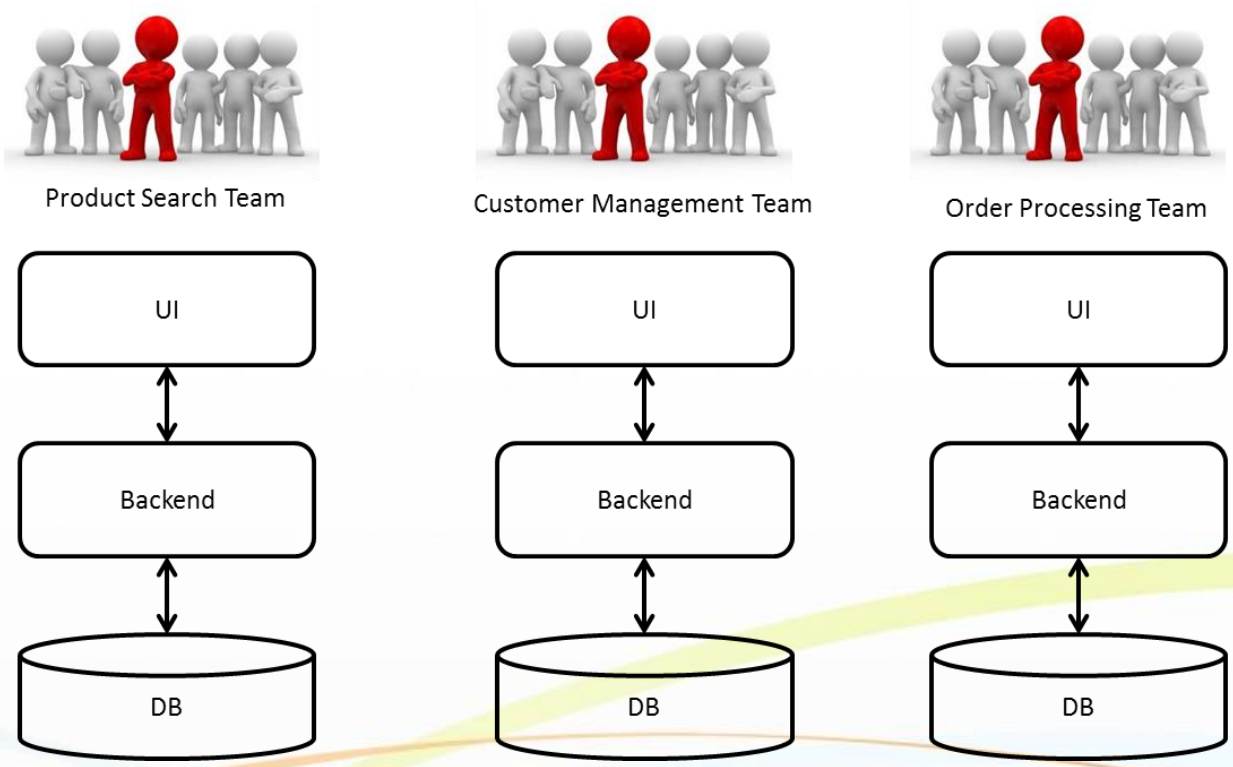
Running on any available set of physical resources (public/private/virtualized)

Microservie Architecture

The Architecture of Legacy Application



The result of microservice introduction

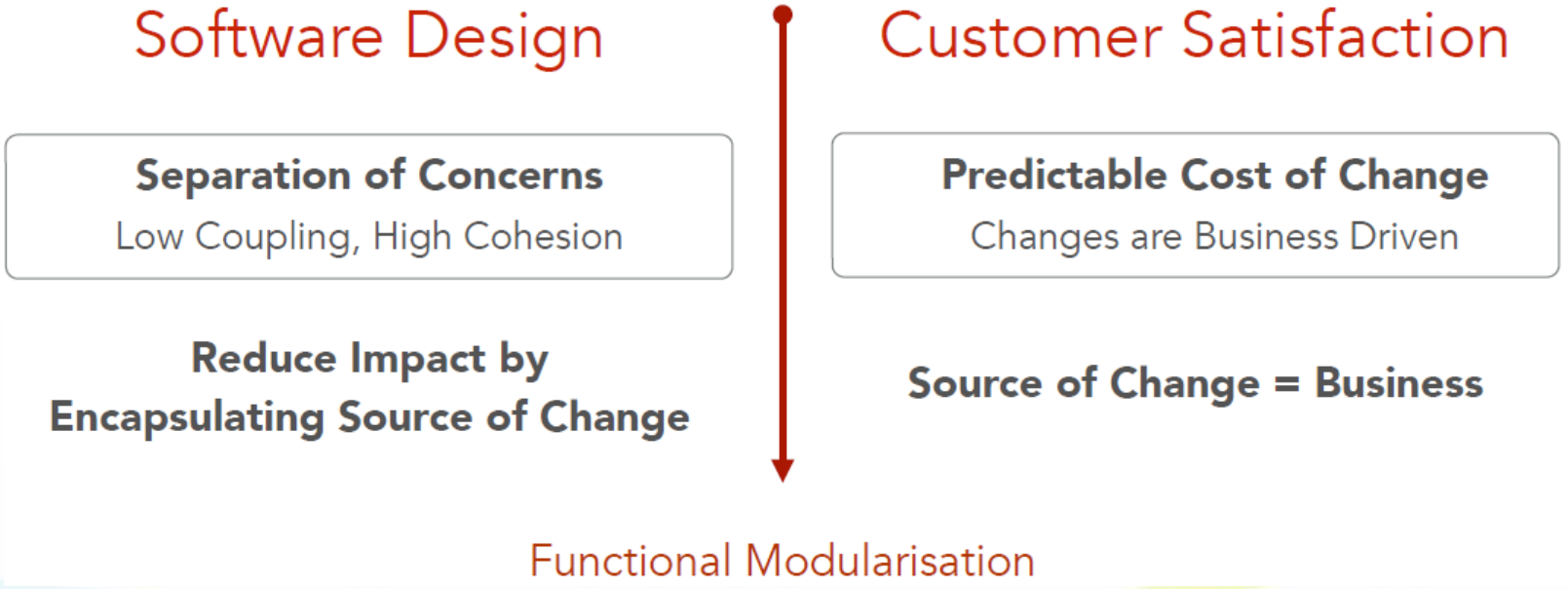


In Wiki, A microservice is

- A microservice (architectrue) is
  - a software development technique—a variant of the service-oriented architecture (SOA) architectural style that structures an application as a collection of loosely coupled services.
- In a microservices architecture,
  - services are fine-grained and
  - the protocols are lightweight.

MicroService Architecture Style

- Structure
  - Functional decomposition of the business domain



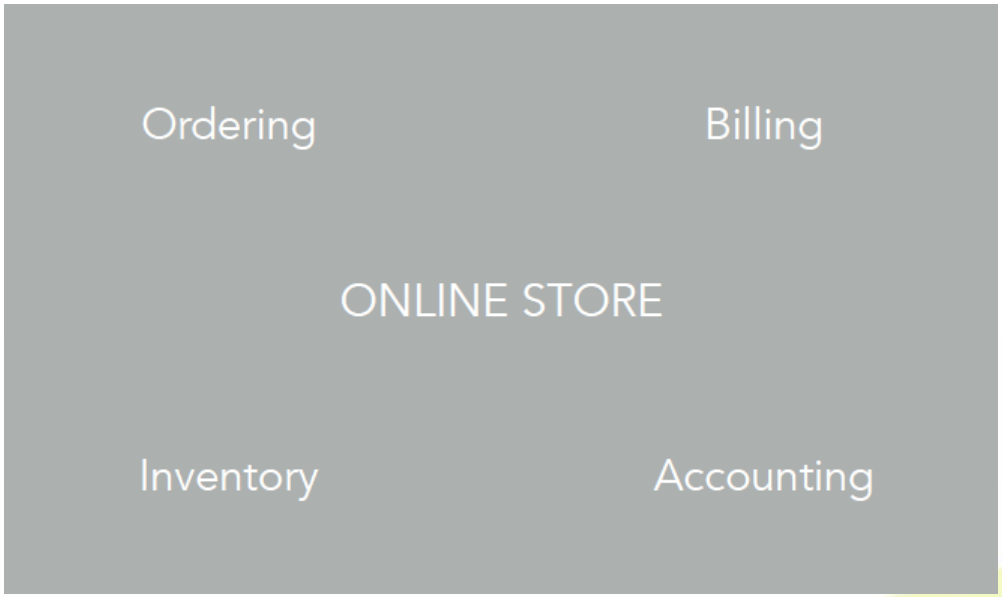
Domain Driven (Decomposition) Design

- Tackling complexity by abstracting the business domain concepts and logic into a domain model and using this as a base for software development
- "In order to create good software, you have to know what that software is all about.
  - You cannot create a banking software system unless you have a good understanding of what banking is all about, one must understand the domain of banking."
- Domain driven design deals with large complex models by dividing them into different functionally bounded subdomains and the explicitly describing the interrelations between these subdomains (Bounded Contexts)

From: Domain Driven Design by Eric Evans

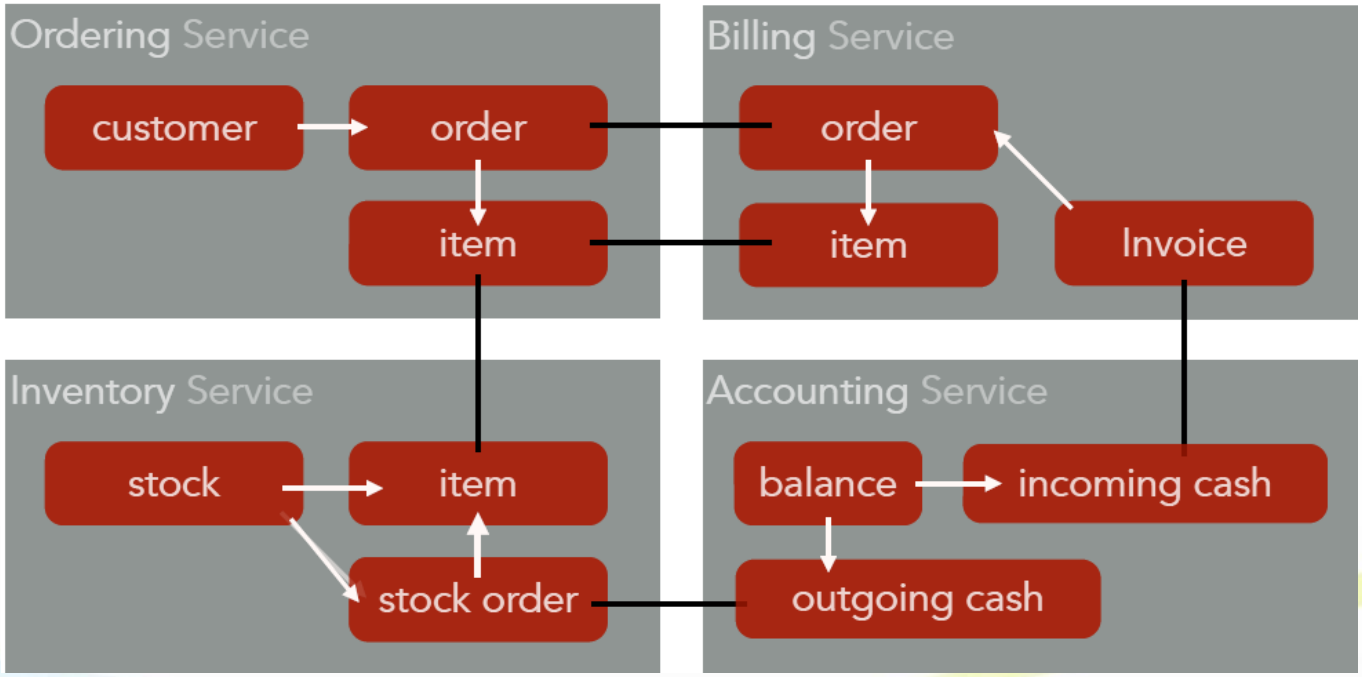
MicroService Architecture Style

- Structure Example
  - Functional decomposition Example of the Online Store business domain



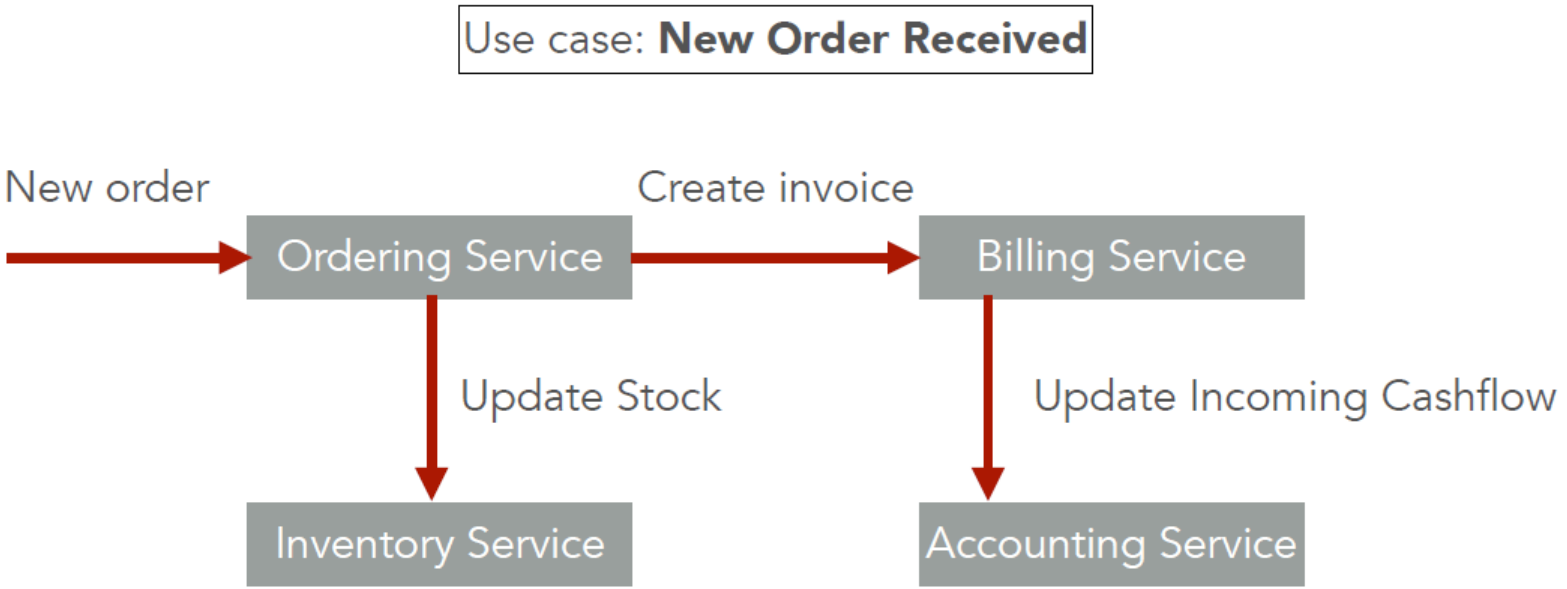
MicroService Architecture Style

- Structure Example
  - Applying services to bounded contexts



MicroService Architecture Style

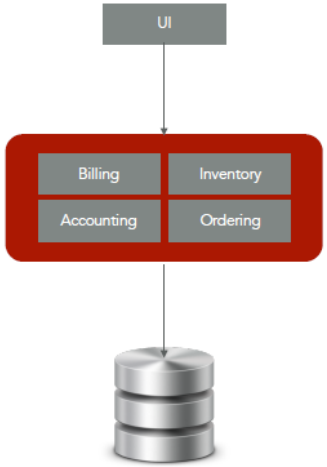
- Behavior Example



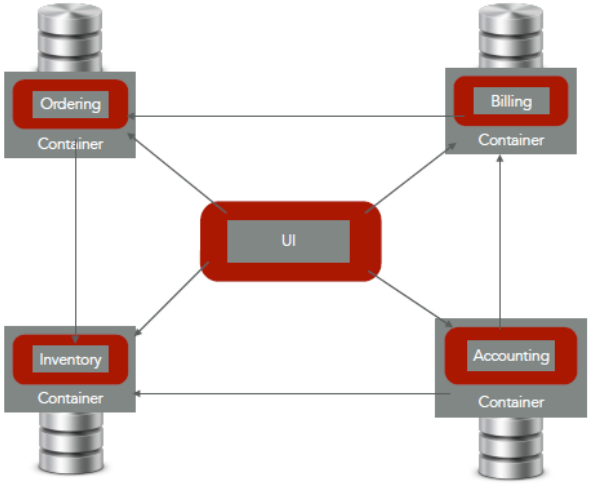
Conway's Law

- Any organization that designs a system (defined broadly) will produce a design whose structure is a copy of the organization's communication structure (Melvyn Conway, 1967)

Technology-based Team Composition



Functional Team Composition



Benefit of DDD, i.e Microservice

- The benefit of decomposing an application into different smaller services is that it improves modularity and makes the application easier to understand, develop, and test and more resilient to architecture erosion.[1]
- It parallelizes development by enabling small autonomous teams to develop, deploy and scale their respective services independently.[2]
- It also allows the architecture of an individual service to emerge through continuous refactoring.[3]
- Microservices-based architectures enable continuous delivery and deployment.[1][4]

MicroService Synapsis

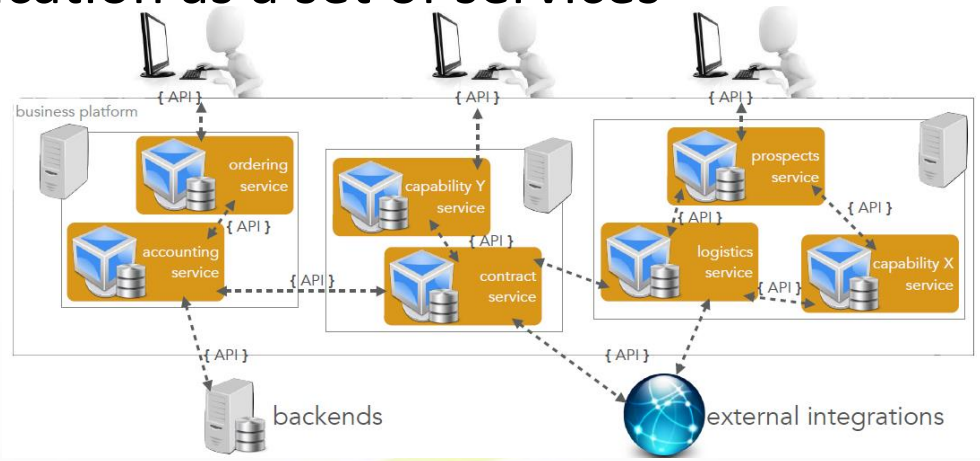
- “Loosely coupled service oriented architecture with bounded context”, [Adrian Cockcroft, April 2015]
- Developing a single application as a suite of small services
  - each running in its own process and communicating with lightweight mechanisms, often in an HTTP API
- Functional decomposition of systems into manageable and independently deployable components, [Andreas Schroeder]
- These services are built around business capabilities and independently deployable by fully automated deployment machinery.

Classic SOA vs. Microservices

- Classic SOA
  - integrates different applications as a set of services

[Stijn Van den Enden et al. @www.aca-it.be]

- Microservices
  - architect a single application as a set of services



Typical **implementation** solution differs!

Classic SOA

integrates different applications as a set of services

Heavy-weight

Orchestration

Intelligent Communication Layer

ESB

WS*/SOAP

License-driven

Target problem:
Integrate (Legacy) Software

Microservices

architect a single application as a set of services

Light-weight

Choreography

Dumb Communication Layer

Intelligent Services

HTTP/REST/JSON

Target problem:
Architect new Business Platform

Features of MicroServices

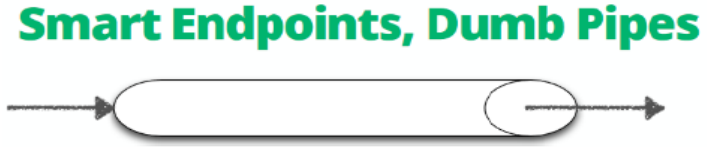
- Functional system decomposition means vertical slicing
  - in contrast to horizontal slicing through layers
- The term “micro” refers to the sizing
  - a microservice must be manageable by a single development team (5-9 developers)
- Services are organized around capabilities.
  - small and focused on doing one thing well
  - In general, one team is responsible for one microservice.
  - It is the size that one agile team can afford.
  - Enabling the independence of microservices.

Features of Microservices

- Independent
  - Changes in one service do not affect other service.
  - Quick fixes and deployment are available.
  - Freedom to choose technologies.
- One team manages everything
  - From development to deployment (DevOps).
  - From UI to Database.
- Independent deployment implies no shared state and inter-process communication
- Improved reusability
  - Services (even deployed) can be shared across projects.

Features of Microservices

- Each microservice only provides an interface.
- Inter-communication between Microservices is often standardized
  - HTTP(S)
    - battle-tested and broadly available transport protocol
  - REST
    - uniform interfaces on data as resources with known manipulation means
  - JSON
    - simple data representation format



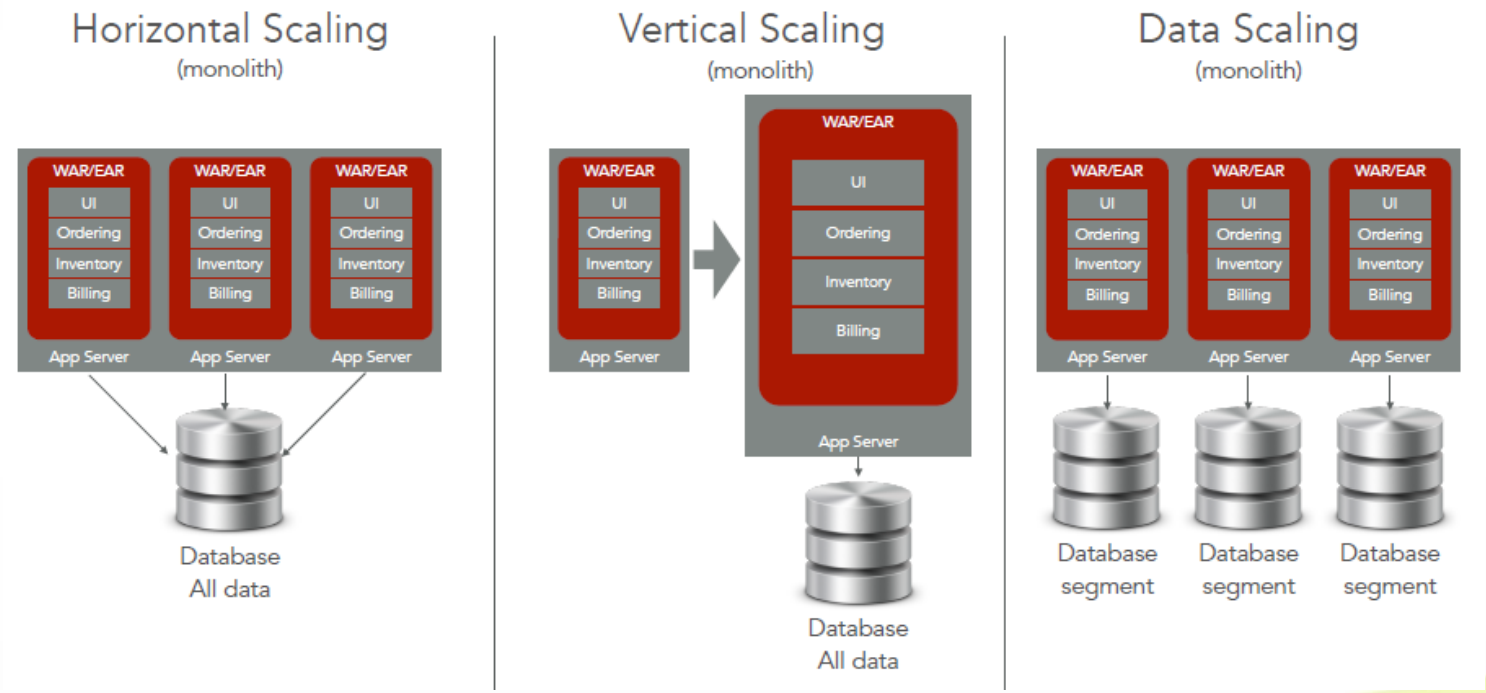
[VirajBrian Wijesuriya @uscs.cmb.ac.lk]

* REST and JSON are convenient because they simplify interface evolution

Features of Microservices

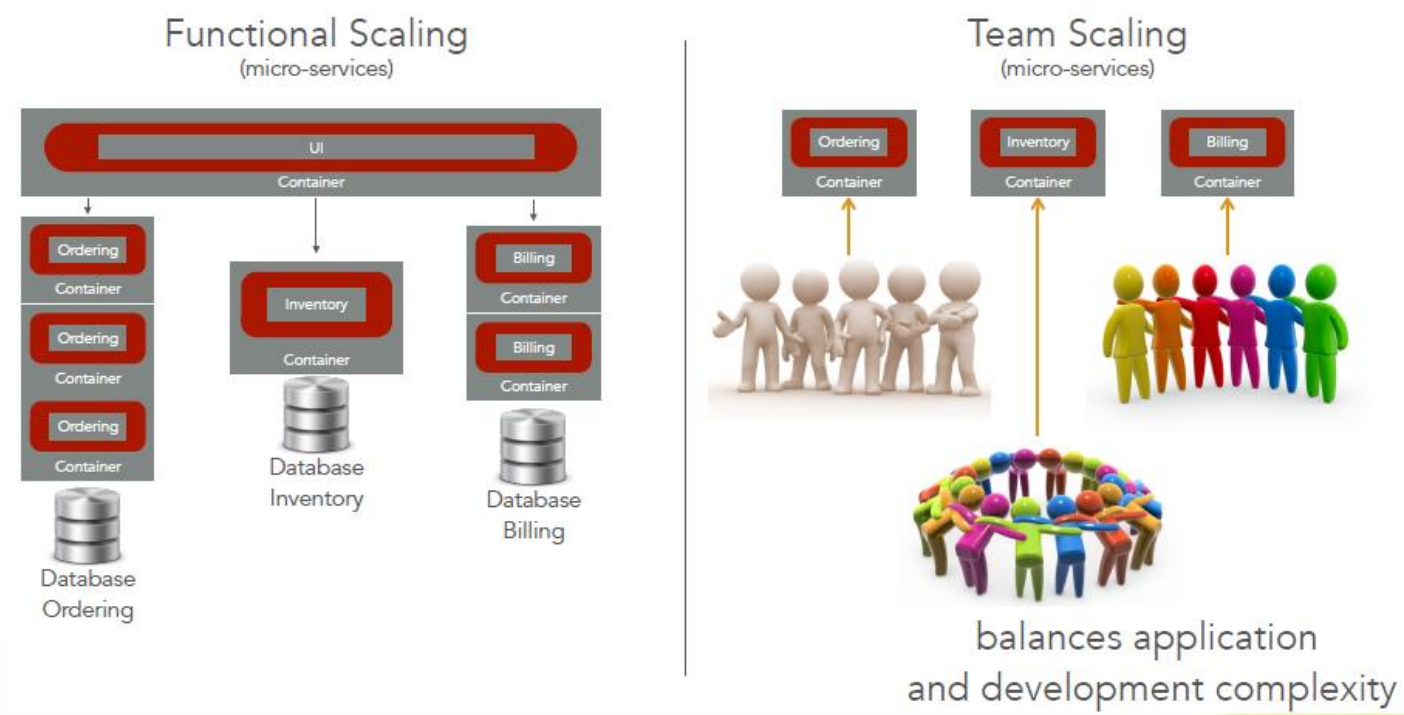
- Better Scaling
  - Each microservice can be scaled independently
  - If there is a heavy load on a particular service, you can expand that service only.
  - Microservices can be extended without affecting other services
  - In the monolithic architecture, it is necessary to increase the total number of servers.

The Limit of Monolith Feature in Classic SOA



[Stijn Van den Enden et al. @www.aca-it.be]

Scaling with MicroServices



[Stijn Van den Enden et al. @www.aca-it.be]

The Challenge

Multiplicity of Stacks

Do services and apps interact appropriately?

Static website

nginx 1.5 + modsecurity + openssl + bootstrap 2

User DB

postgresql + pgv8 + v8

Queue

Redis + redis-sentinel

Analytics DB

hadoop + hive + thrift + OpenJDK

Background workers

Python 3.0 + celery + pyredis + libcurl + ffmpeg + libopencv + nodejs + phantomjs

Web frontend

Ruby + Rails + sass + Unicorn

API endpoint

Python 2.7 + Flask + pyredis + celery + psycopg + postgresql-client

Multiplicity of hardware environments

Development VM

QA server

Customer Data Center

Public Cloud

Production Cluster

Disaster recovery

Production Servers

Contributor's laptop

Can I migrate s moothly and quickly?

The NxM Matrix from Hell

	Development VM	QA Server	Single Prod Server	Onsite Cluster	Public Cloud	Contributor's laptop	Customer Servers
Static website	?	?	?	?	?	?	?
Web frontend	?	?	?	?	?	?	?
Background workers	?	?	?	?	?	?	?
User DB	?	?	?	?	?	?	?
Analytics DB	?	?	?	?	?	?	?
Queue	?	?	?	?	?	?	?

An inspiration and some ancient history! (~1960)

Multiplicity of Goods

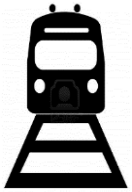


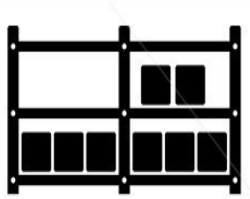




Do I worry about ho w goods interact (e.g . coffee beans next t o spices)



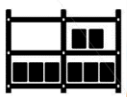



Multipilicity of meth ods for transporting/ storing

Can I transport quickly and smoothly (e.g. from boat to train to truck)

Also an NxM Matrix

	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
							

Solution: Intermodal Shipping Container

Multiplicity of Goods





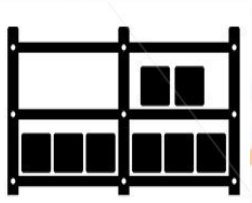


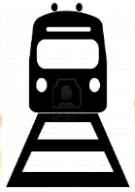
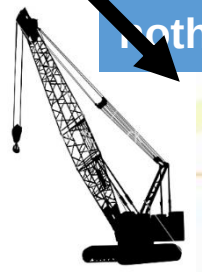
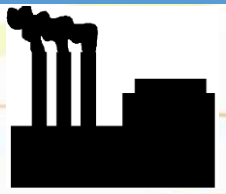
Do I worry about how goods interact (e.g. coffee beans next to spices)

A standard container that is loaded with virtually any goods, and stays sealed until it reaches final delivery.

…in between, can be loaded and unloaded, stacked, transported efficiently over long distances, and transferred from one mode of transport to another

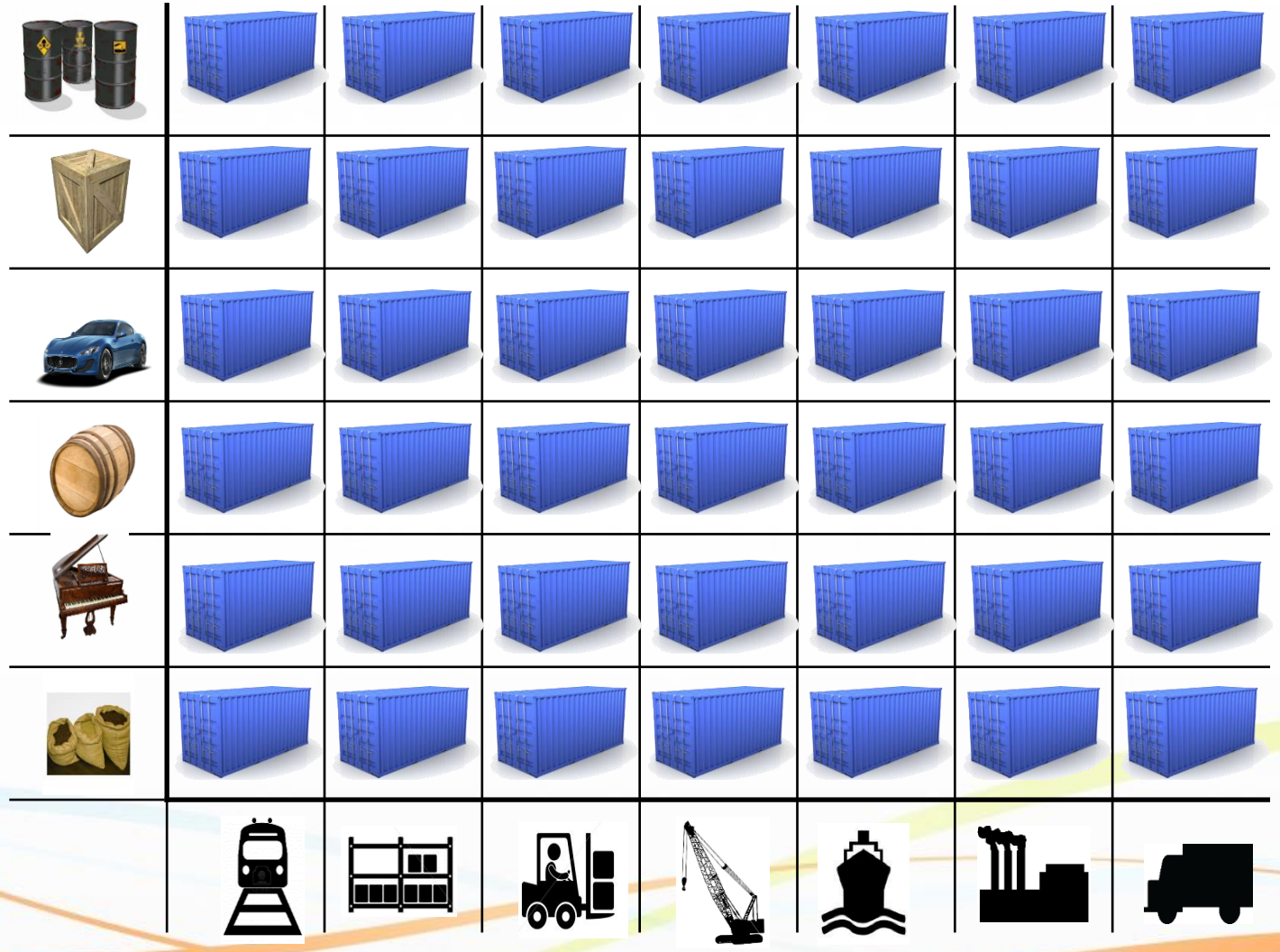
Multiplicity of methods for transporting/storing


Can I transport quickly and smoothly (e.g. from boat to train to truck)

This eliminated the NXN problem…



and spawned an Intermodal Shipping Container Ecosystem

- 90% of all cargo now shipped in a standard container
- **Order of magnitude reduction in cost and time to load and unload ships**
- **Massive reduction in losses** due to theft or damage
- Huge reduction in freight cost as percent of final goods (from >25% to <3%)

→ **massive globalizations**

- 5000 ships deliver 200M containers per year

Docker is a shipping container system for code

Multiplicity of Stacks

Static website | User DB | Web frontend | Queue | Analytics DB

Do services and apps interact appropriately?

An engine that enables any payload to be encapsulated as a lightweight, portable, self-sufficient container…

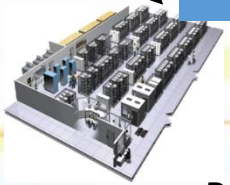
…that can be manipulated using standard operations and run consistently on virtually any hardware platform

Multiplicity of hardware environments


Development VM


QA server


Customer Data Center


Public Cloud


Production Cluster


Contributor's laptop

Can I migrate smoothly and quickly

Or…put more simply

Multiplicity of Stacks

Static website

User DB

Web frontend

Queue

Analytics DB

Do services and apps interact appropriately?

Developer: Build Once, Run Anywhere (Finally)

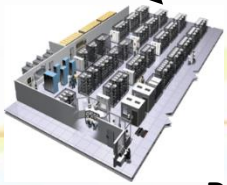
Operator: Configure Once, Run Anything

Multiplicity of hardware environments

Development VM

QA server

Customer Data Center



Public Cloud

Production Cluster

Contributor's laptop

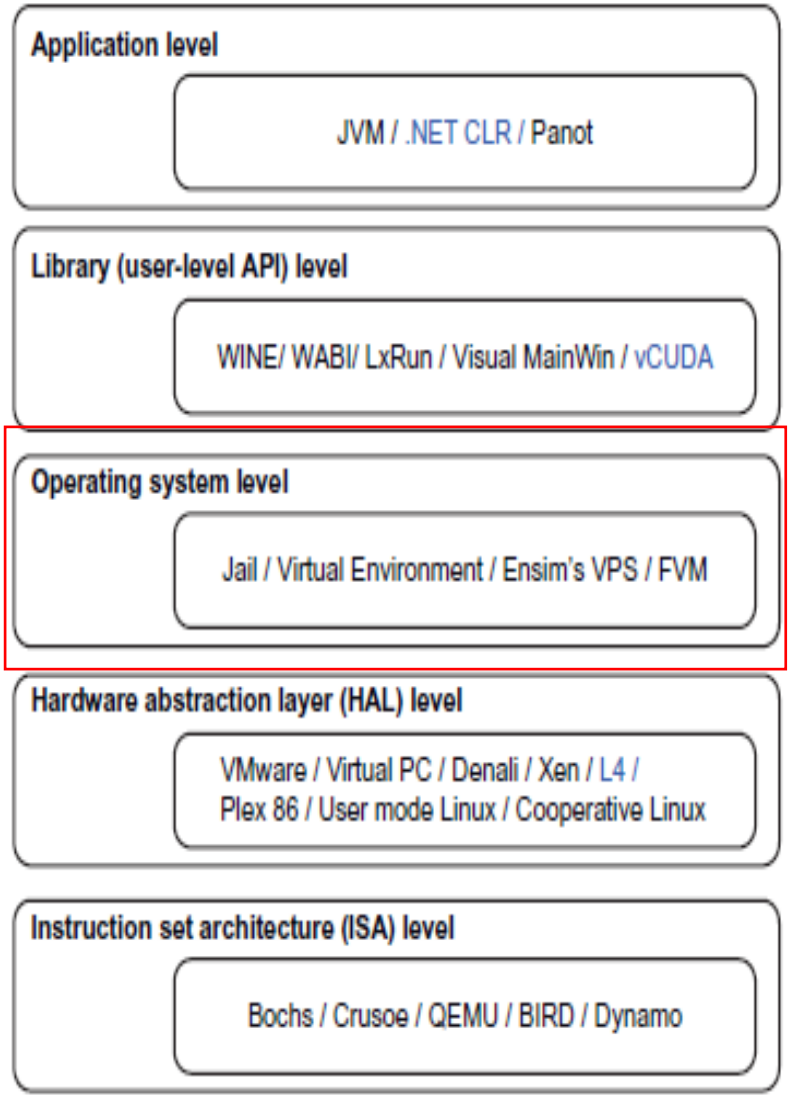
Can I migrate smoothly and quickly

Docker solves the NXN problem

What is Docker

What is Docker

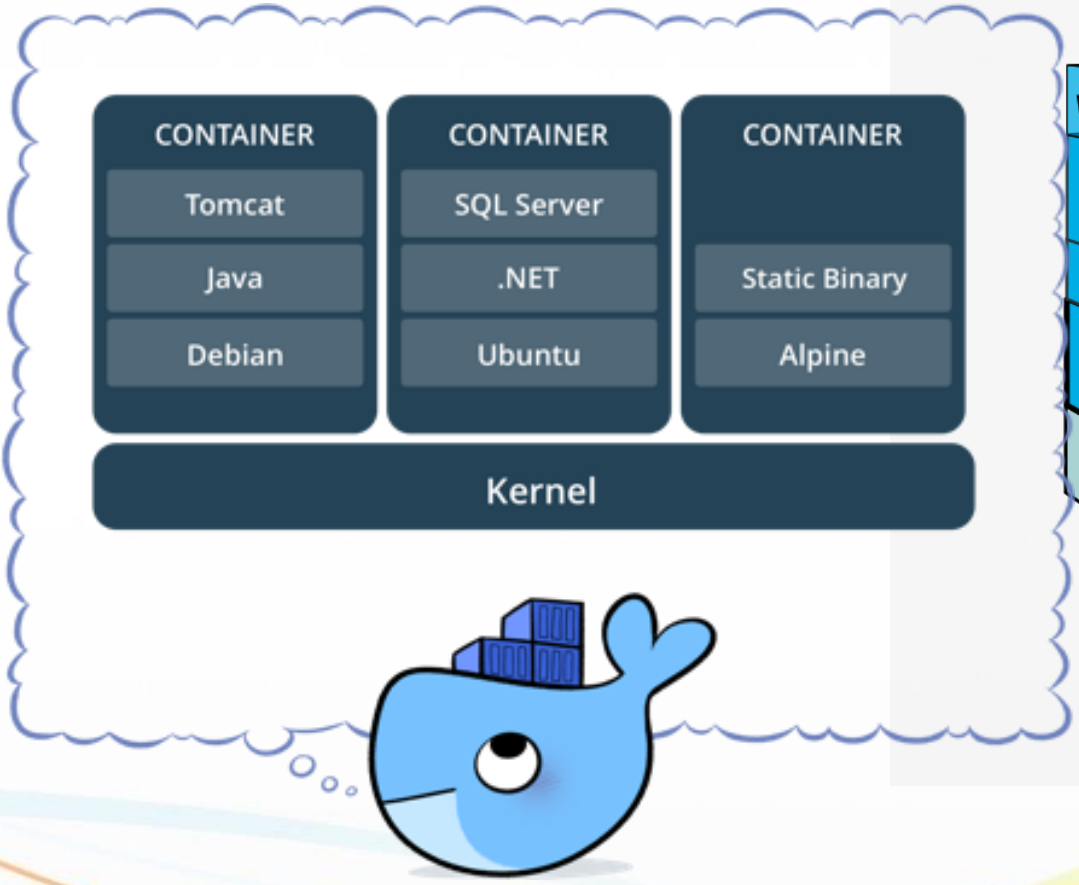
- OS Level Virtualization
  - Refers to an **abstraction layer between traditional OS and user applications**
  - **Creating isolated containers** on a single physical server and the OS instances to utilize the hardware and software in data centers
  - Examples
    - Virtual Execution Environment (VE)
    - Virtual Private Server (VPS)
    - chroot(Change Root)
    - FreeBSD jail
    - LXC, Docker

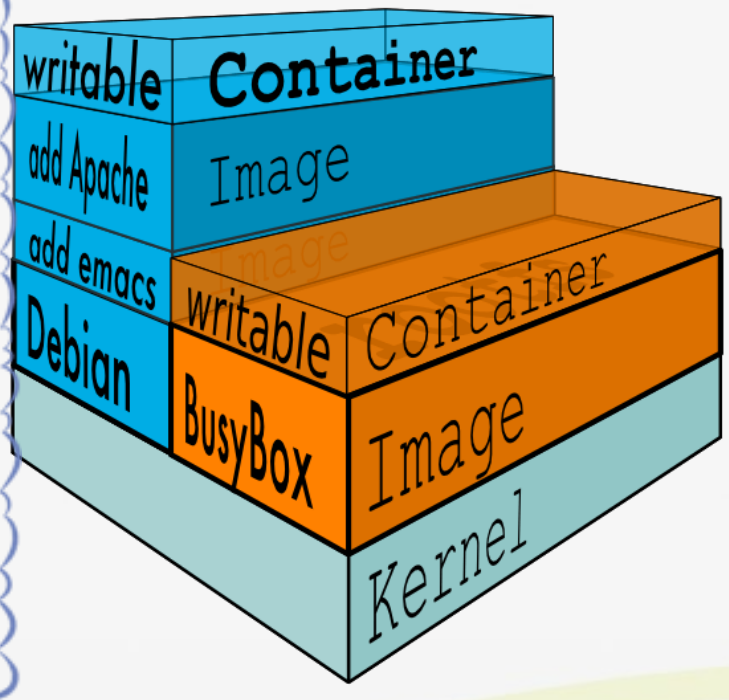


What is Docker

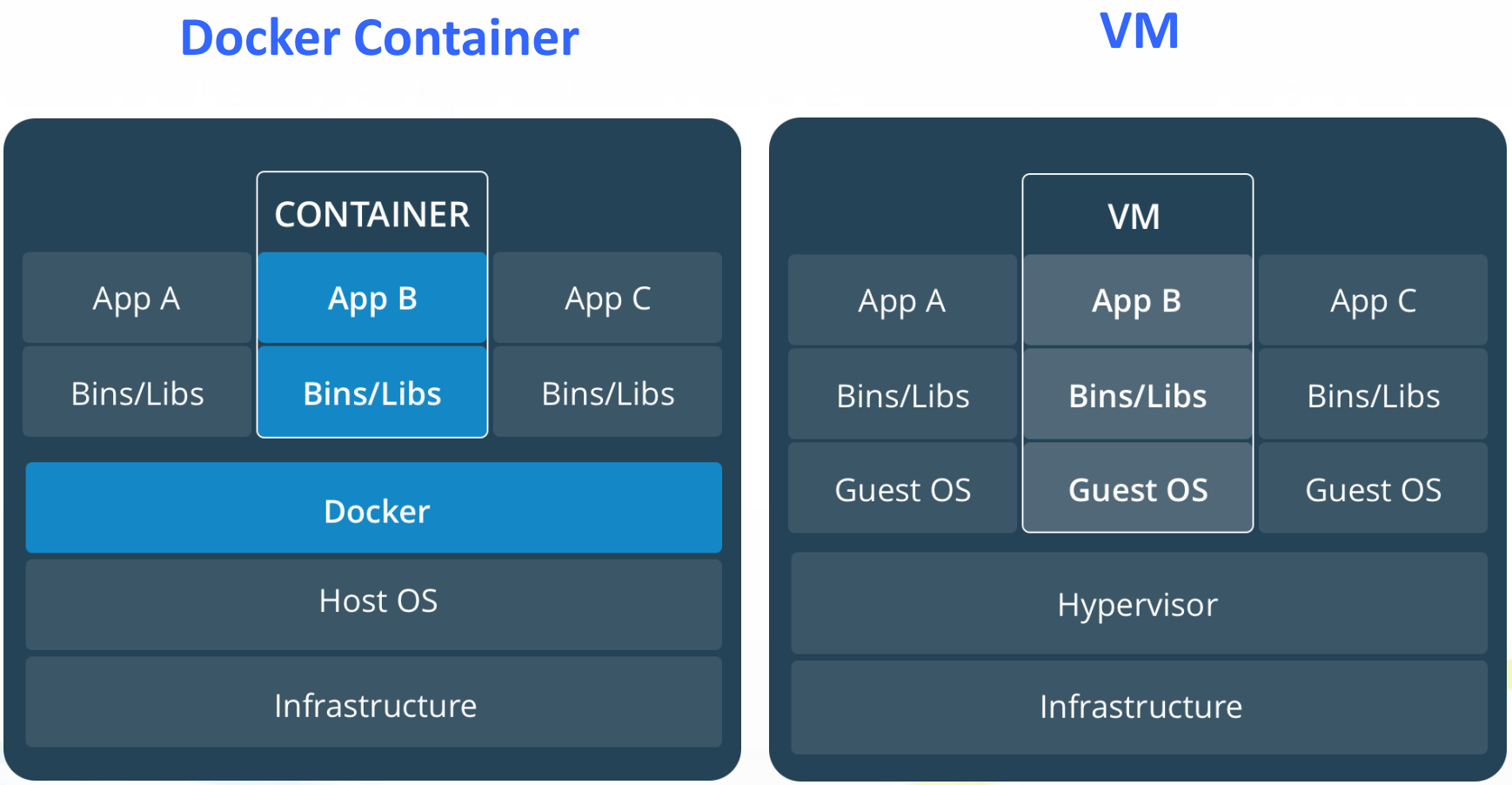
- Docker provides the **ability to package and run an application in** a loosely isolated environment called a **container**.
- The isolation and security allow you to **run many containers simultaneously on a given host**.
- Because of the **lightweight nature of containers**, which run without the extra load of a hypervisor, you can run more containers on a given hardware combination than if you were using virtual machines

Docker Container





Docker Container vs. VM

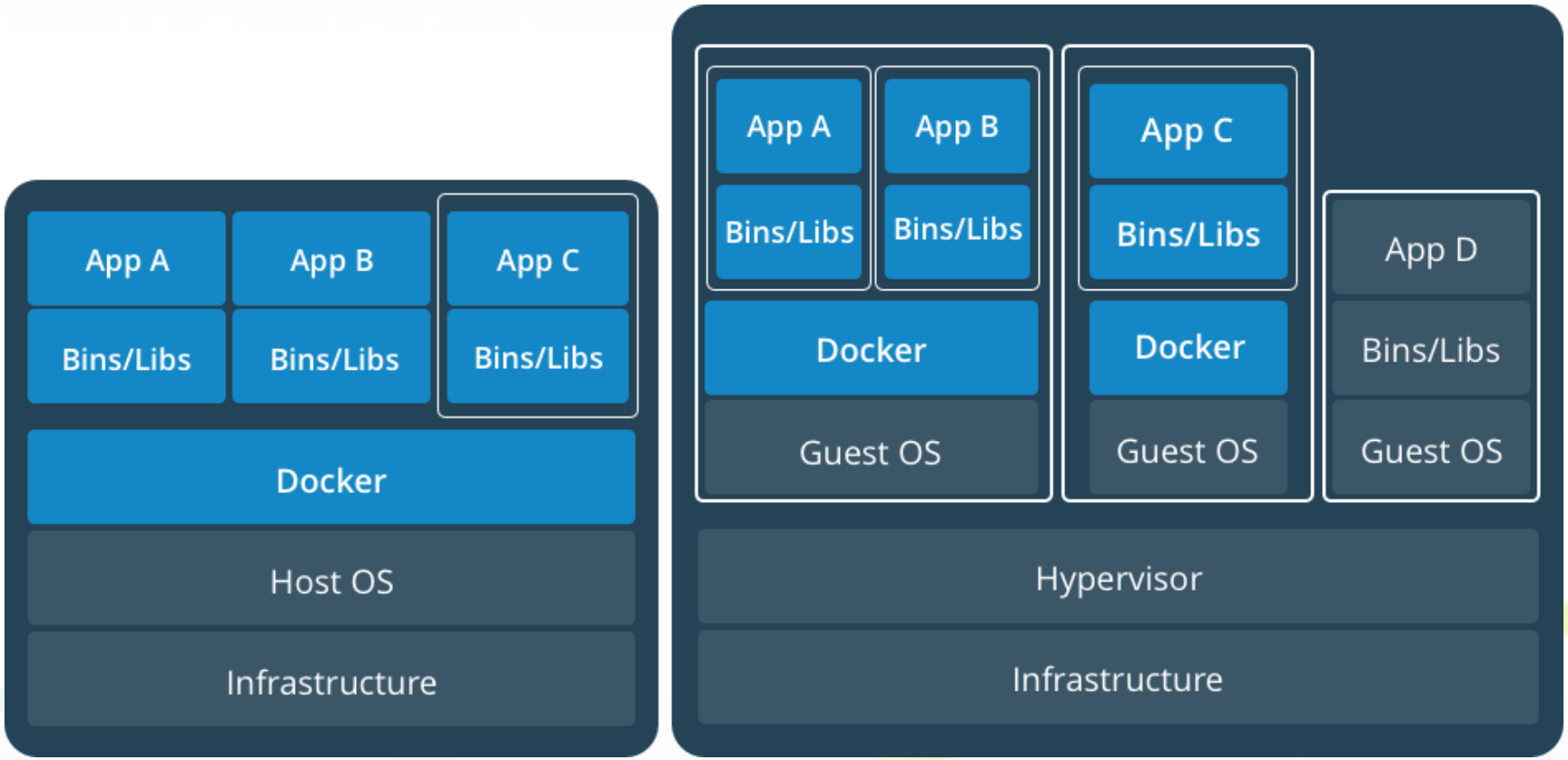


* Containers are isolated, but share OS and, where appropriate bins/libraries

Docker vs VM

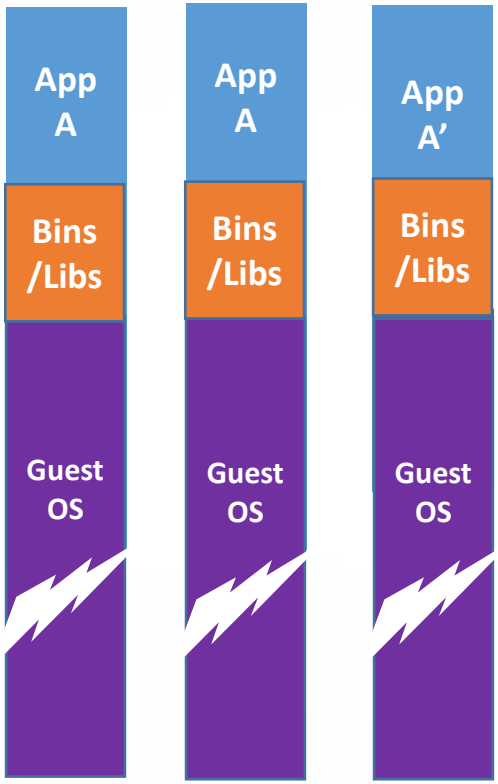
- Virtual machines have a full OS with its own memory management installed with the associated overhead of virtual device drivers. In a virtual machine, valuable resources are emulated for the guest OS and hypervisor, which makes it possible to run many instances of one or more operating systems in parallel on a single machine (or host). Every guest OS runs as an individual entity from the host system.
- On the other hand Docker containers are executed with the Docker engine rather than the hypervisor. Containers are therefore smaller than Virtual Machines and enable faster start up with better performance, less isolation and greater compatibility possible due to sharing of the host's kernel.

Containers and Virtual Machines Together



Why are Docker Containers Lightweight?

VMs



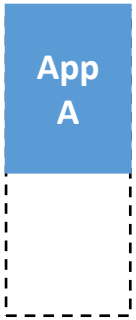
VMs

Every app, every copy of an app, and every slight modification of the app requires a new virtual server

Containers

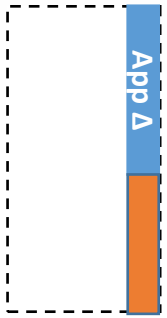
Original App

(No OS to take up space, resources, or require restart)



Copy of App

No OS. Can Share bins/libs



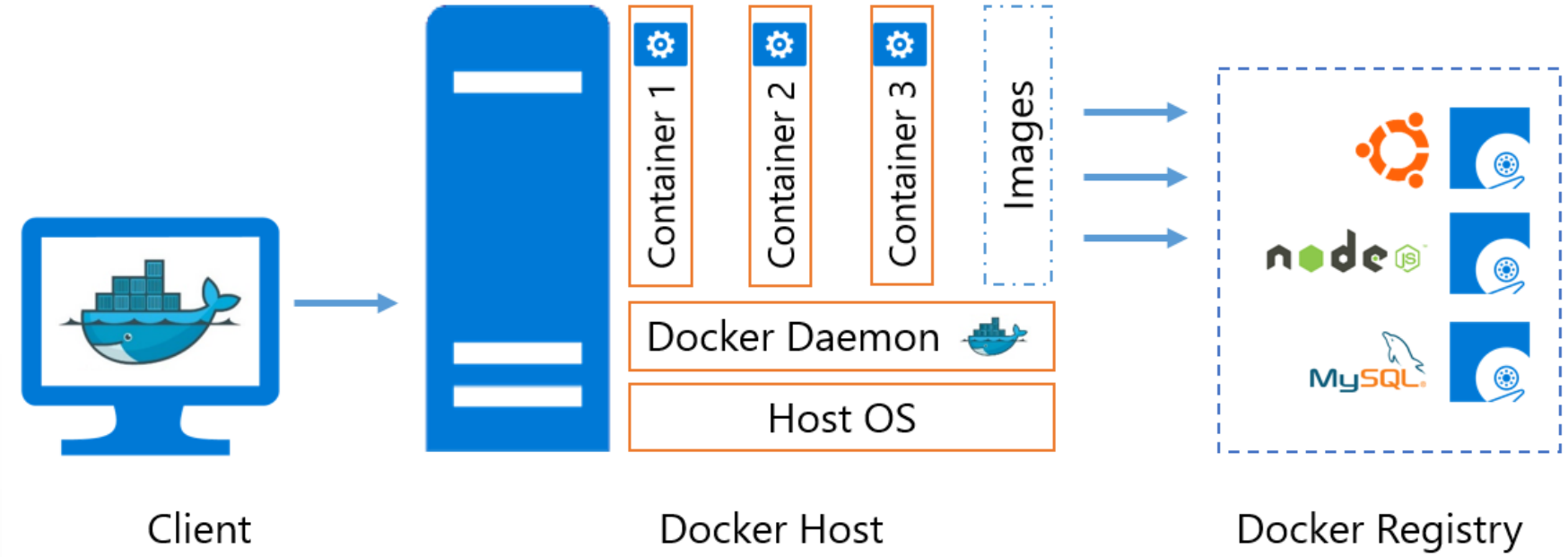
Modified App

Union file system allows us to only save the diffs Between container A and container A'

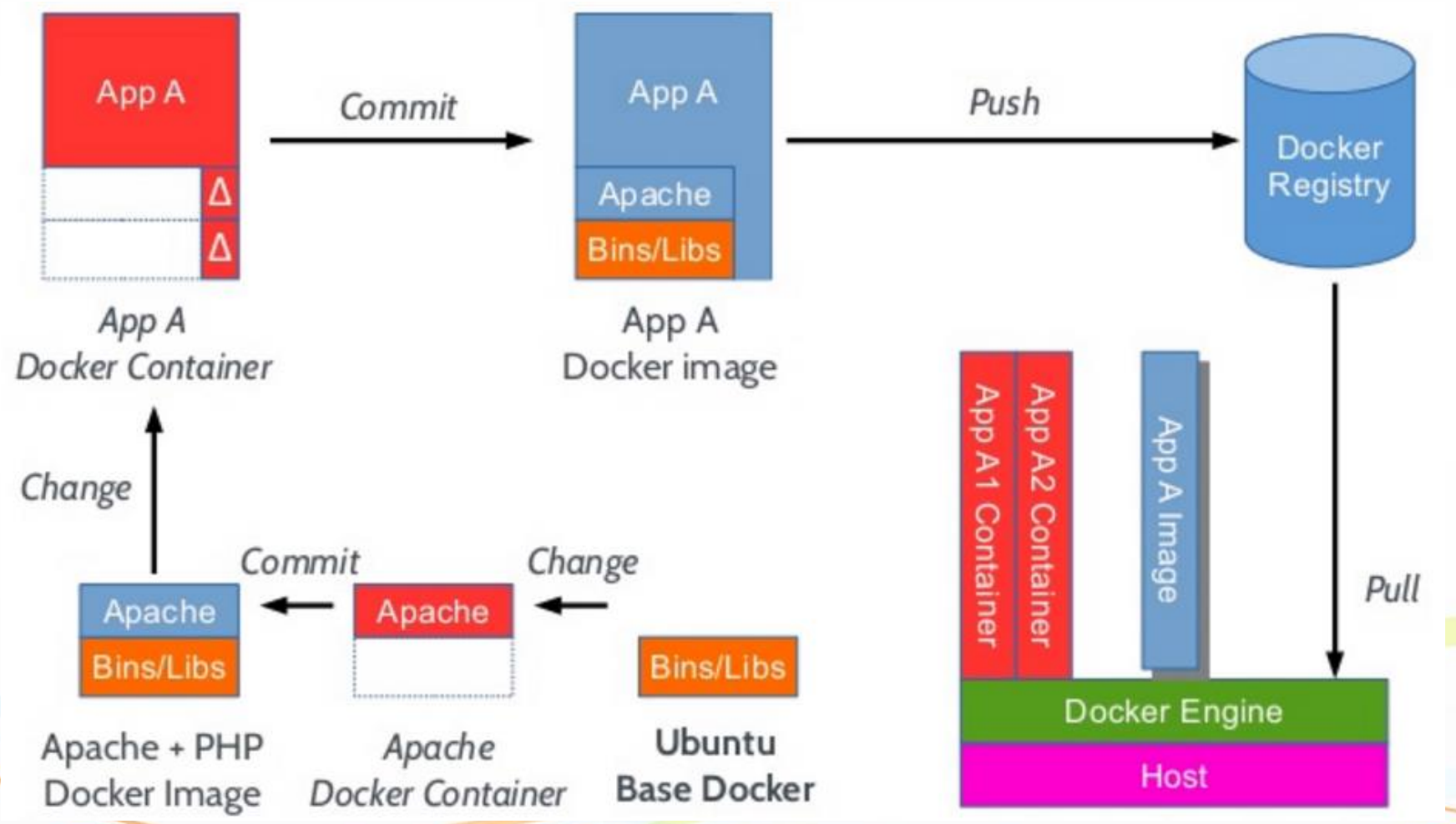
Docker Platform

- Docker provides tooling and a platform to manage the lifecycle of your containers:
  - **Encapsulate your applications (and supporting components) into Docker containers**
  - **Distribute and ship those containers to your teams** for further development and testing
  - **Deploy those applications to your production environment**, whether it is in a local data center or the Cloud

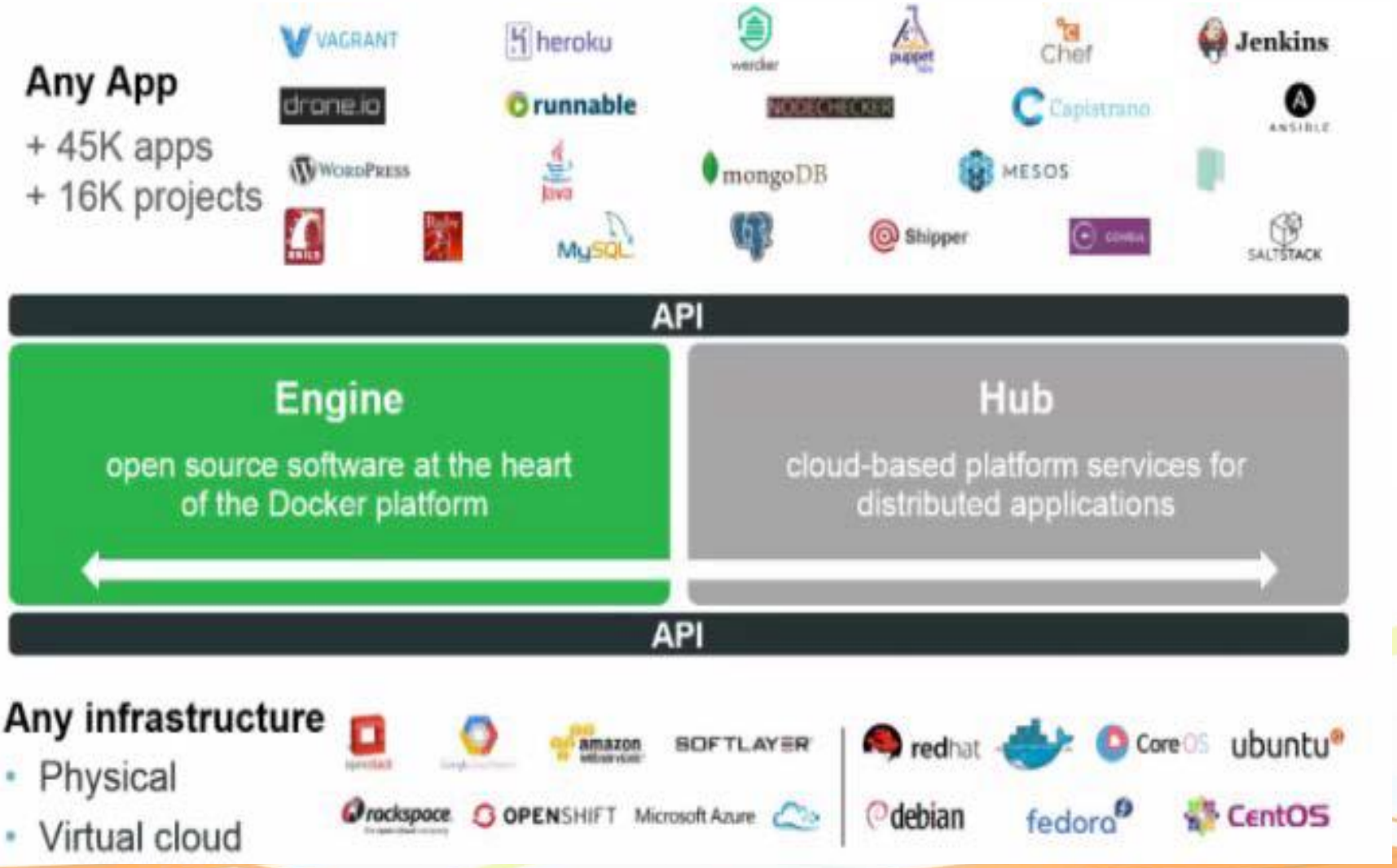
Docker Platform Architecture



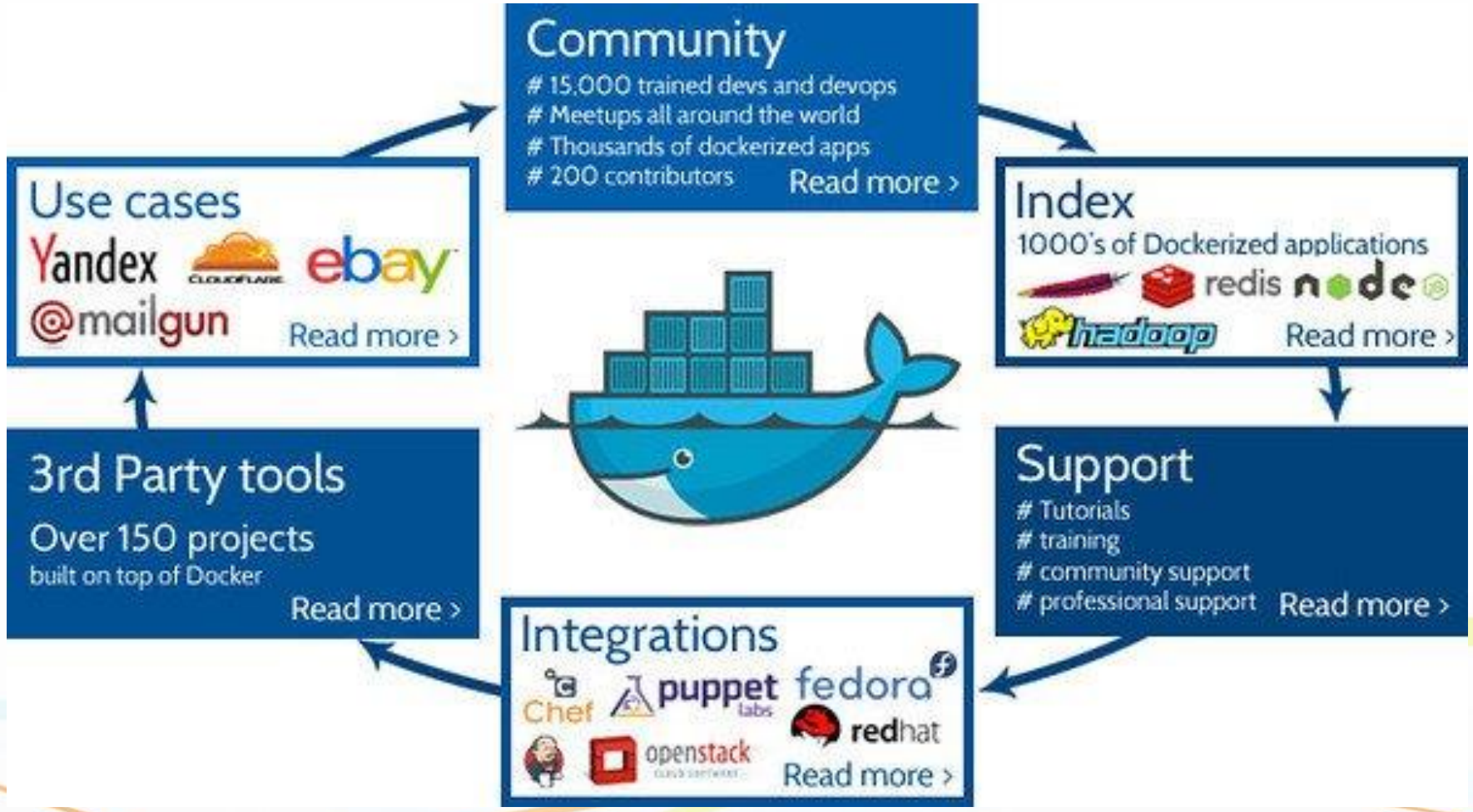
Docker Life Cycle



Docker Ecosystem



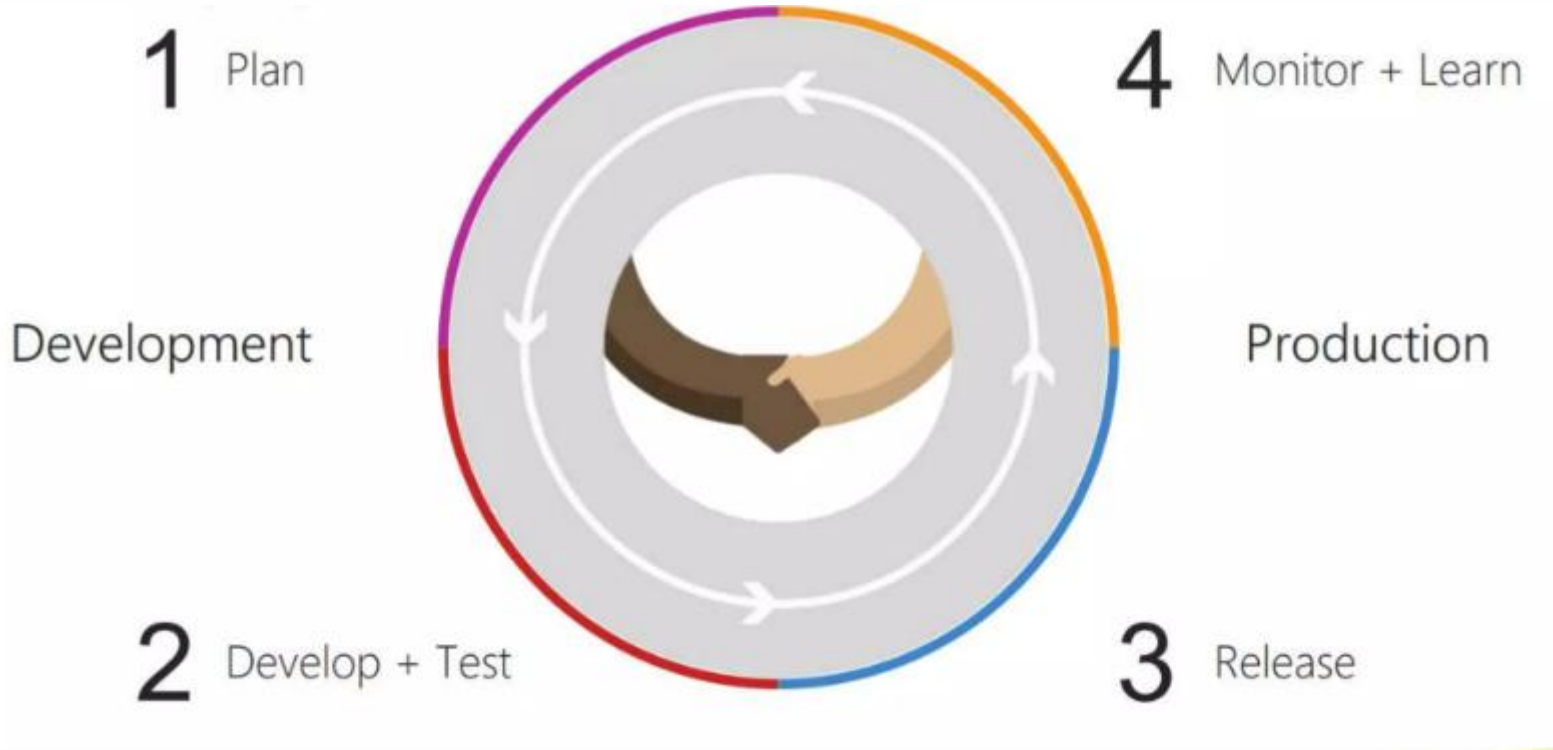
Docker Ecosystem



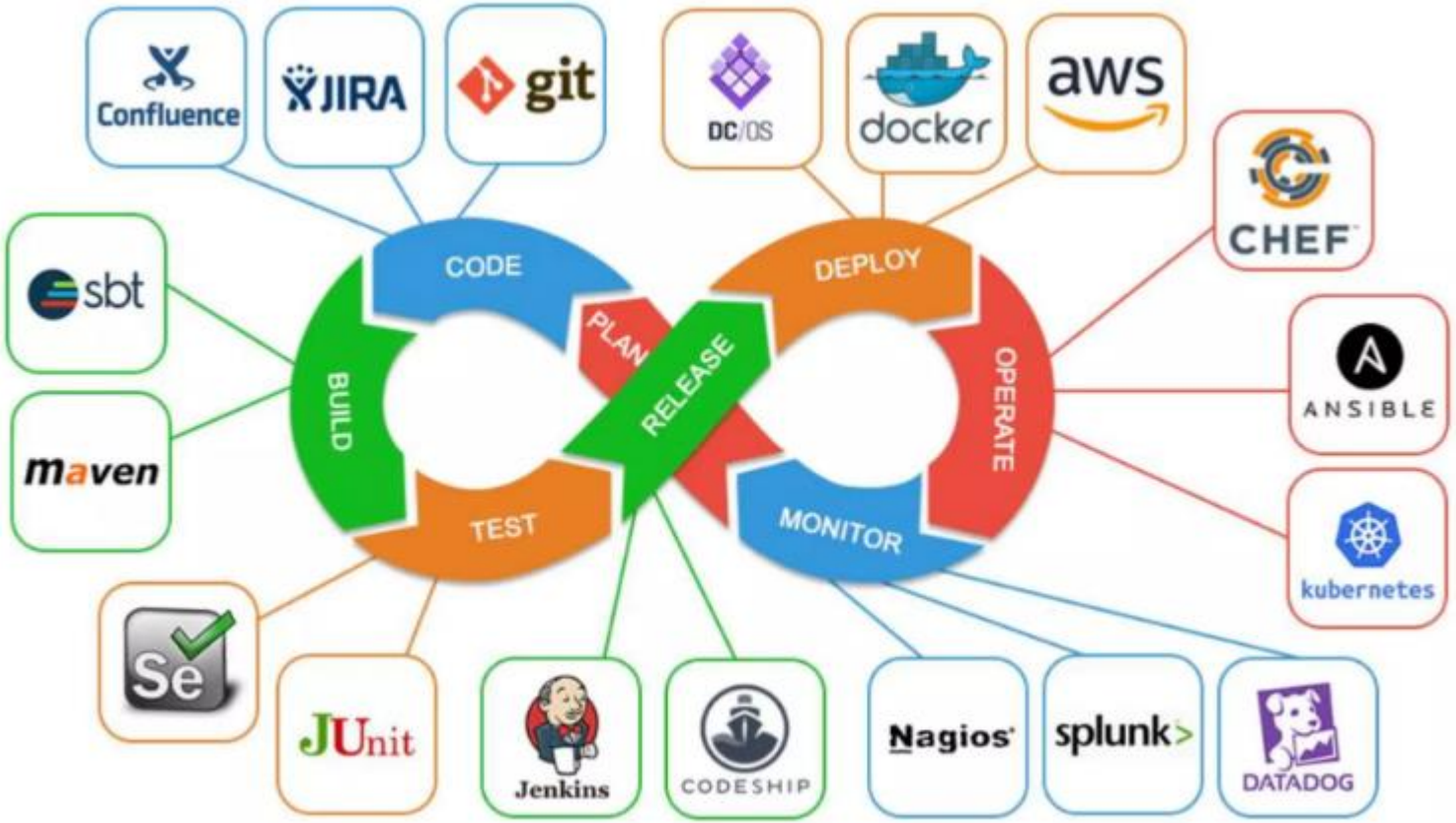
DevOps

- DevOps
  - 소프트웨어 개발(Development)과 IT 운영(Operations)을 결합하여, 개발팀과 운영팀 간의 소통, 협업, 자동화를 통해 더 빠르고 안정적으로 고품질의 소프트웨어를 제공하는 문화 및 방법론
  - 지속적인 통합(CI)과 지속적인 배포(CD)를 가능하게 하고, 팀 효율성 향상, 빠른 릴리스, 품질 및 신뢰성 개선 등의 이점을 제공

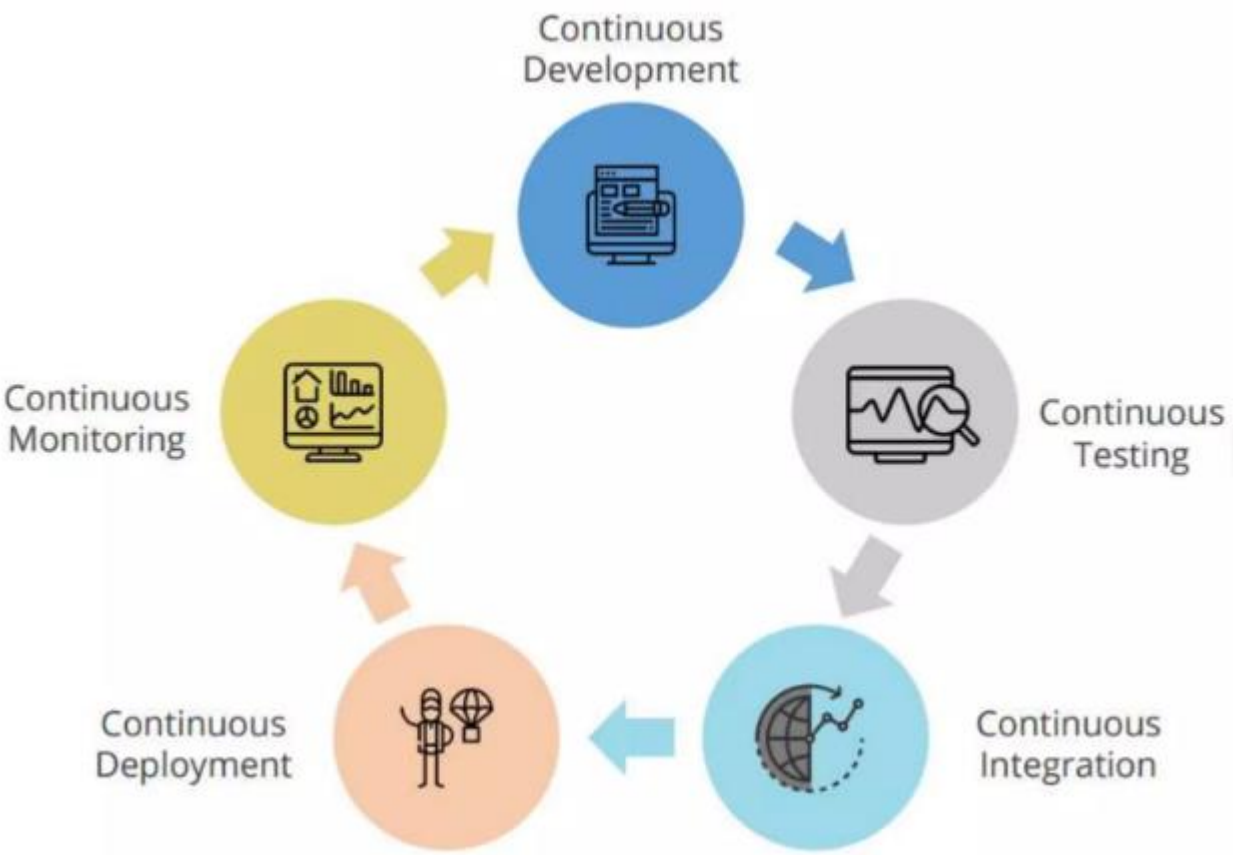
DevOps Process



DevOps ToolChain



DevOps Architecture



CI/CD (Tool: Jenkins, Github Action)

- CI 절차
  - 개발자들이 개발하여 feature 브랜치에 코드를 push합니다.
  - git push를 통해 Trigger되어 CI 서버에서 알아서 Build, Test, Lint를 실행하고 결과를 전송합니다.
  - 개발자들은 결과를 전송받고 에러가 난 부분이 있다면 에러 부분을 수정하고 코드를 master 브랜치에 merge합니다.
  - master 브랜치에 코드를 merge하고 Build, Test가 정상적으로 수행이 되었다면 CI 서버에서 알아서 Deploy 과정을 수행합니다.

DevOps Practices

- Infrastructure as Code
- Continuous integration
- Automated testing
- Application Performance Monitoring / Management
- Continuous Deployment / Delivery
- Release Management
- Configuration Management

DevOps References

- DevOps Foundations (James, Ernest)

  - a three hour set of videos designed to orient beginners into the whole scope of DevOps.

- https://azure.microsoft.com/en-in/overview/devops-tutorial/#understanding
- https://www.udemy.com/course/learn-devops-infrastructure-automation-with-terraform
- http://www.webdevelopmenthelp.net/2017/11/devops-tutorial-for-beginners.html

Benefits/Drawbacks of Docker

Benefits of Docker

- Provisioning
- Performance
- Density

Provisioning

- ***Fast**, consistent delivery of your applications*
  - Docker can streamline the development lifecycle by allowing developers to work in standardized environments using local containers which provide your applications and services. You can also integrate Docker into your continuous integration and continuous deployment (CI/CD) workflow.

Performance

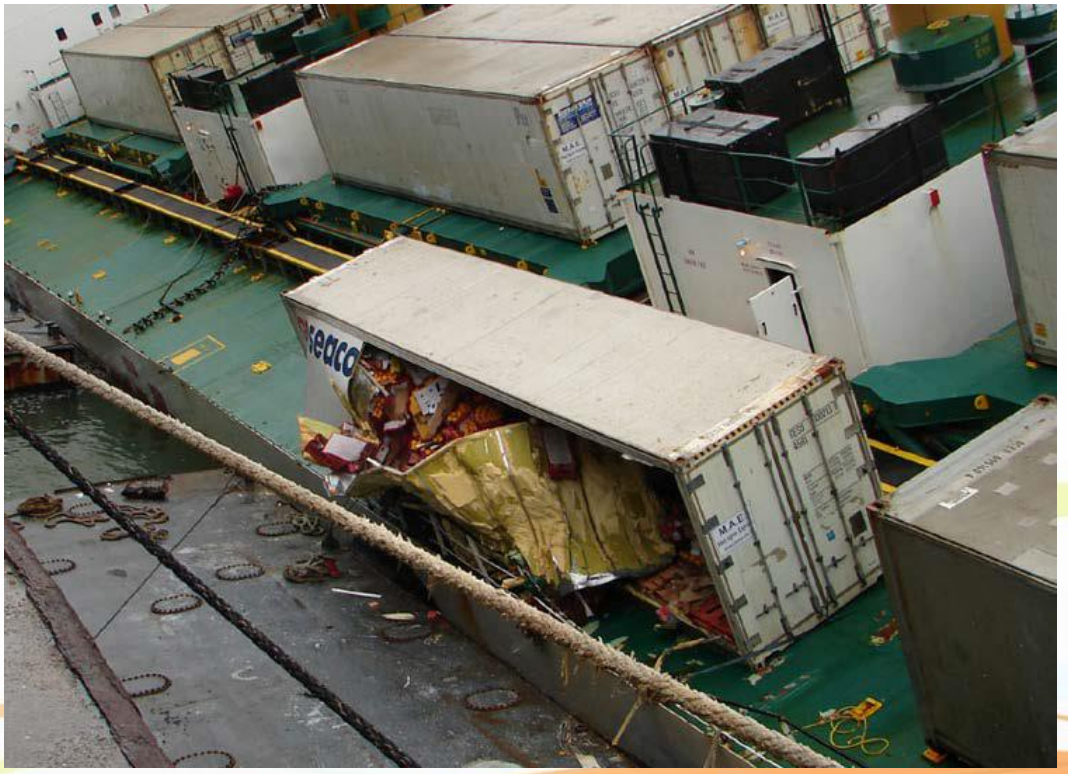
- ***Responsive deployment** and **scaling***
  - Docker's container-based platform allows for highly portable workloads. Docker containers can run on a developer's local host, on physical or virtual machines in a data center, in the Cloud, or in a mixture of environments.
  - Docker's portability and lightweight nature also make it easy to dynamically manage workloads, scaling up or tearing down applications and services as business needs dictate, in near real time.

Density

- ***Running more workloads*** *on the same hardware*
  - Docker is lightweight and fast. It provides a viable, cost-effective alternative to hypervisor-based virtual machines, allowing you to use more of your compute capacity to achieve your business goals.
  - This is useful in high density environments and for small and medium deployments where you need to do more with fewer resources.

Drawbacks of Docker

- Reduced isolation
- Reduced security



Contents

- Why Docker ?
  - Docker is a deploying container system for code
- What is Docker ?
  - Lightweighted Container to package and run an application
  - Docker containers are isolated, but share OS and appropriate bins/libraries
- Docker Platform Architecture
  - Docker Client, Docker Engine, Docker (Hub) Registry
- Docker Lice Cycle
  - Pull, Change, Commit, Push
- Docker Eco System
  - Many Dockerized Applications, Use Cases, Projects
  - Technical Support, 3rd Party Tools, Communites
- Benefits/Drawbacks of Docker
  - Benefits: Fast Provisioning, Continuous Integration/Deployment, Scale up/down, High Dencity (can run more workload on the same HW)
  - Drawbacks: Reduced Isolaton and Security

Thank You!!

Any Questions ??